

# E-Commerce Order & Sales Monitoring System

## Case Overview

An e-commerce company operates multiple product category stores. Each store generates a daily CSV transaction file recording every order — its status, category, and amount. The operations team needs an automated shell script that processes these files each morning, flags problem stores, and outputs a structured CSV sales summary report.

## Input: Order CSV File Format

Order files are stored in /orders, one CSV file per store category per day. Each line is a comma-separated record with five fields:

| Order ID | Date       | Status    | Category    | Amount (₹) |
|----------|------------|-----------|-------------|------------|
| ORD001   | 2025-06-10 | COMPLETED | electronics | 4500.00    |
| ORD002   | 2025-06-10 | FAILED    | clothing    | 1200.00    |
| ORD003   | 2025-06-10 | PENDING   | electronics | 8900.00    |
| ORD004   | 2025-06-10 | REFUNDED  | grocery     | 350.00     |
| ORD005   | 2025-06-10 | COMPLETED | clothing    | 2750.00    |

```
/orders/electronics.csv /orders/clothing.csv /orders/grocery.csv
/reports/sales_YYYY-MM-DD.csv
/alert_logs/alert_log.txt
```

## Task 01 Environment Setup

Define all directory paths and configuration values as variables at the top of your script.

|             |                                                  |
|-------------|--------------------------------------------------|
| ORDER_DIR   | Directory containing daily .csv order files      |
| REPORT_DIR  | Directory where CSV reports will be saved        |
| ALERT_LOG   | Path to the alert log file                       |
| DATE        | Today's date — used in filenames and log entries |
| REPORT_FILE | Full path for today's CSV report                 |

### Your Code — Write Below

```
#!/bin/bash
```

```
ORDER_DIR=
REPORT_DIR=
ALERT_LOG=
DATE=$(date +%Y-%m-%d)
REPORT_FILE=$REPORT_DIR/sales_$DATE.csv
```

## Task 02 Input File Validation

Before processing, verify the order directory exists, contains .csv files, and that each file is non-empty. Log an alert and stop if any check fails.

### Your Code — Write Below

```
validate_files() {
    # Check if ORDER_DIR exists

    # Check if .csv files are present

    # Check each file is non-empty

    # Log alert and exit if any check fails
}
```

## Task 03 CSV File Processing — Nested Loops

Use a for loop to iterate over each .csv file in ORDER\_DIR. Inside each iteration, use a while read loop with IFS set to comma to parse each CSV row into separate fields.

### Your Code — Write Below

```
for csv_file in $ORDER_DIR/*.csv; do

    while IFS=',' read -r order_id date status category amount; do
        [ "$order_id" = "order_id" ] && continue

        # Your row processing here

    done < "$csv_file"

done
```

## Task 04 Filter Orders Using grep

Use grep to extract rows matching specific order statuses from each CSV file. This is faster than reading line by line for simple pattern matching.

### Your Code — Write Below

```
failed_count=$( ... "$csv_file")
pending_count=$( ... "$csv_file")
refund_count=$( ... "$csv_file")

echo "Failed: $failed_count | Pending: $pending_count | Refunded: $refund_count"
```

## Task 05 Column Extraction & Sorting Using cut and sort

Use cut to extract specific columns from a CSV file. Use sort to order the results. Combine both to find the highest-value orders.

Your Code — Write Below

```
# Extract the amount column from all COMPLETED orders
amounts=$(... ',COMPLETED,' "$csv_file" | cut ...)

# Sort amounts highest to lowest
top_orders=$(... ',COMPLETED,' "$csv_file" | cut ... | sort ...)

# Print top 5 orders by value
echo "$top_orders" | ...
```

Task 06 Revenue Aggregation Using AWK

Use awk to total the revenue from COMPLETED orders and calculate the average order value for each store file.

Your Code — Write Below

```
# Calculate total revenue from COMPLETED orders
revenue=$(... ',' ... "$csv_file")

# Count total orders (skip header)
total=$(...',' ... "$csv_file")

# Calculate average order value
avg=$(... ',' ... "$csv_file")
```

Task 07 Alert Detection Logic

Assign a health status to each store based on its failed order count and refund rate. Use if/elif/else conditional statements.

|               |                                           |
|---------------|-------------------------------------------|
| FAILED > 30   | Status = CRITICAL — log alert immediately |
| REFUNDED > 20 | Status = WARNING — log warning            |
| Otherwise     | Status = OK — log info                    |

Task 08 Generate CSV Report

Write the daily summary as a structured CSV file so it can be opened in any spreadsheet tool. Use echo with >> to build the file row by row.

Expected CSV output:

| Store/Category | Revenue   | Orders | Failed | STATUS   |
|----------------|-----------|--------|--------|----------|
| electronics    | ₹1,42,500 | 318    | 12     | OK       |
| clothing       | ₹58,200   | 205    | 31     | WARNING  |
| grocery        | ₹21,800   | 189    | 42     | CRITICAL |

Task 09 Maintain Alert Log

Record every important event to ALERT\_LOG with a timestamp and severity label. This gives the operations team a clear audit trail.

```
[2025-06-10 06:00:00] [INFO]      Script started
[2025-06-10 06:00:02] [INFO]      Processing electronics.csv
[2025-06-10 06:00:04] [CRITICAL]  grocery: 42 failed orders detected
[2025-06-10 06:00:05] [INFO]      CSV report written: sales_2025-06-10.csv
```

### Task 10 Implement Functions

Refactor all logic into named functions. Call them in sequence from the main section of your script.

|                             |                                                   |
|-----------------------------|---------------------------------------------------|
| <code>validate_files</code> | Check ORDER_DIR and CSV files before processing   |
| <code>process_store</code>  | Run grep, cut, sort, awk analysis on one CSV file |
| <code>generate_csv</code>   | Write the full CSV report from collected results  |
| <code>write_alert</code>    | Append a timestamped entry to ALERT_LOG           |
| <code>show_menu</code>      | Display the interactive menu (Task 11)            |

### Task 11 Menu System Using Case Statement

Add an interactive menu so the operations team can run specific actions on demand without editing the script.

#### Your Code — Write Below

```
show_menu() {
    echo "1. Process today's orders"
    echo "2. Generate CSV report"
    echo "3. View alert log"
    echo "4. Exit"
    read -p "Select [1-4]: " choice
    case $choice in
        1) ... ;; 2) ... ;; 3) cat "$ALERT_LOG" ;; 4) exit 0 ;; *) echo "Invalid"
    ;;
    esac
}
```

### Task 12 Cron Job Scheduling

Schedule the script to run automatically every morning at 06:00 AM so the operations team has the report ready at the start of each workday.

|                |                         |
|----------------|-------------------------|
| <b>MIN</b>     | 0 (at minute 0)         |
| <b>HOURL</b>   | 6 (6 AM, 24-hour clock) |
| <b>DAY</b>     | * (every day)           |
| <b>MONTH</b>   | * (every month)         |
| <b>WEEKDAY</b> | * (every day of week)   |